

Basic Information

- **Title:** Bank Account Management System using Inheritance
-

Objective / Aim

Aim:

To write a Java program to demonstrate **inheritance, method overriding, constructors, encapsulation of account operations, and polymorphic behavior** using a Bank Account Management System.

The program creates a base Account class and derives SavingsAccount and CurrentAccount classes from it. It performs operations such as **deposit, withdrawal, interest calculation, fund transfer, and displaying account details**.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 Processor or equivalent
- Minimum 4 GB RAM
- Minimum 10 GB free storage

Software

- Operating System: Windows/Linux/macOS
 - Java Development Kit: JDK 17 or JDK 21
 - IDE/Text Editor: VS Code / Eclipse / IntelliJ IDEA / NetBeans
 - Command Prompt / Terminal
-

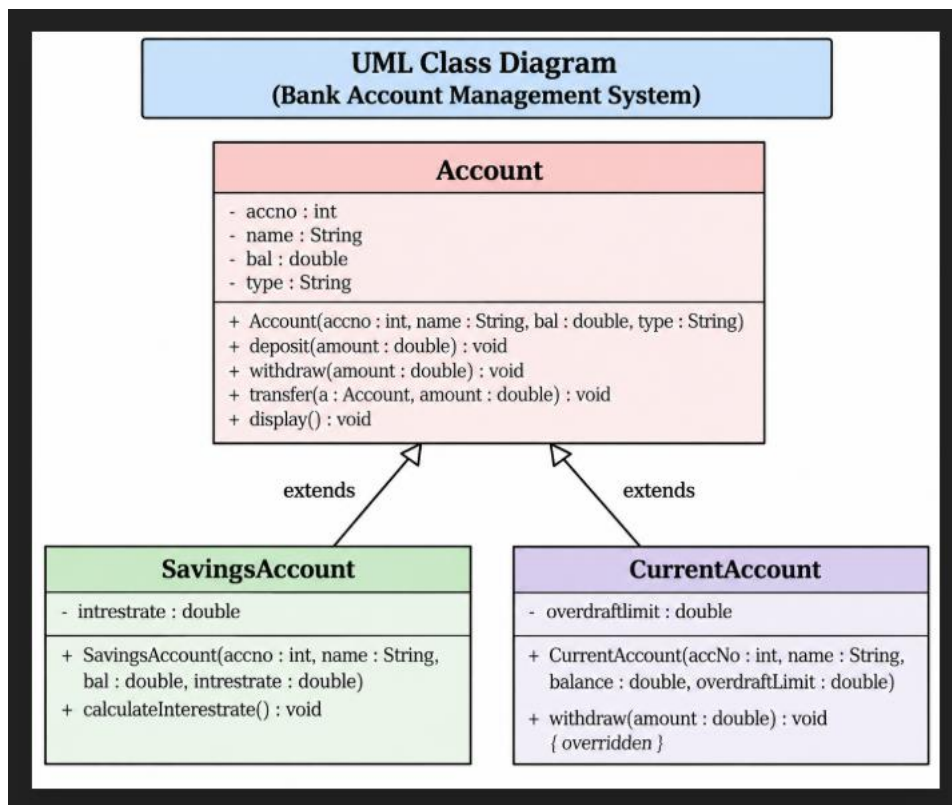
Algorithm / Logic Description

1. Start the program.
2. Create a base class named Account.
3. Declare account number, account holder name, balance, and account type.
4. Create a constructor to initialize the account details.
5. Define the deposit() method to add money to the account.

6. Define the `withdraw()` method to withdraw money if sufficient balance is available.
 7. Define the `transfer()` method to transfer money from one account to another.
 8. Define the `display()` method to display account details.
 9. Create a `SavingsAccount` class that extends the `Account` class.
 10. Add an interest rate to the savings account.
 11. Define `calculateInterestRate()` to calculate the interest.
 12. Create a `CurrentAccount` class that extends the `Account` class.
 13. Override the `withdraw()` method to support the overdraft limit.
 14. In the `main()` method, create objects of `SavingsAccount` and `CurrentAccount`.
 15. Display the initial account details.
 16. Perform deposit, withdrawal, interest calculation, and transfer operations.
 17. Display the final account details and balances.
 18. Stop the program.
-

UML Class Diagram

You can draw the following diagram in your record:



Relationship:

`SavingsAccount` and `CurrentAccount` **inherit** from `Account` using the `extends` keyword.

Source Code

Your code can be written as follows:

```
class Account
```

```
{
```

```
    int accno;
```

```
    String name;
```

```
    double bal;
```

```
    String type;
```

```
    Account(int accno, String name, double bal, String type)
```

```
{
```

```
    this.accno = accno;
```

```
    this.name = name;
```

```
this.bal = bal;
this.type = type;
}

// Deposits money into the account
void deposit(double amount)
{
    bal = bal + amount;

    System.out.println("Deposited amount is: " + amount);
    System.out.println("Available balance: " + bal);
}

// Withdraws money from the account
void withdraw(double amount)
{
    if (bal < amount)
    {
        System.out.println("Insufficient balance");
    }

    if (amount <= bal)
    {
        bal = bal - amount;

        System.out.println("Amount withdrawn is: " + amount);
        System.out.println("Remaining balance is: " + bal);
    }
}
```

```

// Transfers money from one account to another
void transfer(Account a, double amount)
{
    if (amount <= bal)
    {
        bal = bal - amount;
        a.bal = a.bal + amount;

        System.out.println("Transferred amount is: " + amount);
    }
    else
    {
        System.out.println("Insufficient account balance");
    }
}

// Displays account details
void display()
{
    System.out.println("Account Number: " + accno);
    System.out.println("Account holders name: " + name);
    System.out.println("Account type: " + type);
    System.out.println("Account balance: " + bal);
}
}

// Child class of Account

```

```

class SavingsAccount extends Account
{
    double intrestrate;

    SavingsAccount(int accno, String name, double bal, double intrestrate)
    {
        super(accno, name, bal, "savings");
        this.intrestrate = intrestrate;
    }

    // Calculates interest
    void calculateInterestrte()
    {
        double interest;

        interest = bal * intrestrate / 100;

        System.out.println("Interest is: " + interest);
    }
}

```

```

// Child class of Account
class CurrentAccount extends Account
{
    double overdraftlimit;

    CurrentAccount(int accNo, String name, double balance,
        double overdraftLimit)

```

```

{
    super(accNo, name, balance, "Current");
    this.overdraftlimit = overdraftLimit;
}

// Method overriding
@Override
void withdraw(double amount)
{
    if (amount <= bal + overdraftlimit)
    {
        bal = bal - amount;

        System.out.println("Withdraw = " + amount);
    }
    else
    {
        System.out.println("Over Limit Exceeded");
    }
}
}

```

```

// Main class
public class BankAccountManagement
{
    public static void main(String[] args)
    {
        SavingsAccount s =

```

```
new SavingsAccount(1018, "sai", 50000, 5);
```

```
CurrentAccount c =
```

```
new CurrentAccount(1019, "Riya", 10000, 3000);
```

```
System.out.println("Before transaction\n");
```

```
s.display();
```

```
System.out.println();
```

```
c.display();
```

```
System.out.println();
```

```
System.out.println("Transactions are:");
```

```
s.deposit(50000);
```

```
s.withdraw(20000);
```

```
s.calculateInterestrates();
```

```
c.deposit(40000);
```

```
c.withdraw(20000);
```

```
s.transfer(c, 20000);
```

```
System.out.println("\nAfter Transactions\n");
```

```
s.display();
```

```

        System.out.println();

        c.display();

        System.out.println("\nFinal Balance\n");

        System.out.println("Savings: " + s.bal);
        System.out.println("Current: " + c.bal);
    }
}

```

Important concepts used

Concept	Used in your program
Class	Account, SavingsAccount, CurrentAccount
Object	s, c
Constructor	Account(), SavingsAccount(), CurrentAccount()
Inheritance	extends Account
super	Calls parent constructor
this	Refers to current object
Method	deposit(), withdraw(), display()
Method Overriding	CurrentAccount.withdraw()
@Override	Indicates overridden method
Encapsulation concept Account data and operations grouped in classes	

Compilation & Execution Commands

Save the program **exactly** as:

BankAccountManagement.java

Compile

```
javac BankAccountManagement.java
```

Run

```
java BankAccountManagement
```

Output Screen

The output for your given values will be approximately:

Before transaction

Account Number: 1018

Account holders name: sai

Account type: savings

Account balance: 50000.0

Account Number: 1019

Account holders name: Riya

Account type: Current

Account balance: 10000.0

Transactions are:

Deposited amount is: 50000.0

Available balance: 100000.0

Amount withdrawn is: 20000.0

Remaining balance is: 80000.0

Interest is: 4000.0

Deposited amount is: 40000.0

Available balance: 50000.0

Withdraw = 20000.0

Transferred amount is : 20000.0

After Transactions

Account Number: 1018

Account holders name: sai

Account type: savings

Account balance: 40000.0

Account Number: 1019

Account holders name: Riya

Account type: Current

Account balance: 50000.0

Final Balance

Savings: 40000.0

Current: 50000.0

Result

Thus, the Java program for the Bank Account Management System was successfully implemented and executed using inheritance, constructors, method overriding, and account operations such as deposit, withdrawal, interest calculation, and money transfer.